

main.c

Run ▶

Output

```
1  "keyword">#include "string"><stdio.h>
2  "keyword">#include "string"><stdlib.h>
3  "keyword">#include "string"><pthread.h>
4  "keyword">#include "string"><unistd.h>
5  "keyword">#define NUM_PHILOSOPHERS 5
6
7  pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
8
9  "keyword">"keyword">void* philosopherLifeCycle("keyword">"keyword">void* arg) {
10     "keyword">int id = *(("keyword">int*)arg);
11     "keyword">int left_chopstick = id;
12     "keyword">int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
13     "keyword">while (1) {
14         // Think
15         pr"keyword">intf("Philosopher %d is thinking...\n", id);
16         // Pick up chopsticks
17         pthread_mutex_lock(&chopsticks[left_chopstick]);
18         pthread_mutex_lock(&chopsticks[right_chopstick]);
19         // Eat
20         pr"keyword">intf("Philosopher %d is eating...\n", id);
21         sleep(rand() % 3 + 1); // Eating time
22         // Put down chopsticks
23         pthread_mutex_unlock(&chopsticks[left_chopstick]);
24         pthread_mutex_unlock(&chopsticks[right_chopstick]);
25         // Repeat the cycle
26     }
```

Philosopher 0 is thinking...
Philosopher 0 is eating...
Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 2 is eating...
Philosopher 3 is thinking...
Philosopher 4 is thinking...
Philosopher 0 is thinking...
Philosopher 1 is eating...
Philosopher 3 is eating...
Philosopher 2 is thinking...
Philosopher 4 is eating...
Philosopher 1 is thinking...
Philosopher 0 is eating...

=== Code Execution Successful ===

```
28 "keyword">int main() {
29     pthread_t philosophers[NUM_PHILOSOPHERS];
30     "keyword">int philosopher_ids[NUM_PHILOSOPHERS];
31     // Initialize mutex locks
32     "keyword">for ("keyword">int i = 0; i < NUM_PHILOSOPHERS; ++i) {
33         pthread_mutex_init(&chopsticks[i], "keyword">NULL);
34     }
35     // Create philosopher threads
36     "keyword">for ("keyword">int i = 0; i < NUM_PHILOSOPHERS; ++i) {
37         philosopher_ids[i] = i;
38         pthread_create(&philosophers[i], "keyword">NULL, philosopherLifeCycle, ("keyword">"keyword">void*)&philosopher_ids[i]);
39     }
40     // Wait for threads to finish (although they run indefinitely)
41     "keyword">for ("keyword">int i = 0; i < NUM_PHILOSOPHERS; ++i) {
42         pthread_join(philosophers[i], "keyword">NULL);
43     }
44     // Destroy mutex locks
45     "keyword">for ("keyword">int i = 0; i < NUM_PHILOSOPHERS; ++i) {
46         pthread_mutex_destroy(&chopsticks[i]);
47     }
48     "keyword">return 0;
49 }
50 }
```